

Tenancy Module

Multi-Tenant Control Plane & Database-per-Tenant Routing

AlfacodeTeam PhpServicePlatform

Gated Demand Architecture

PHP 8.2+

solves: `tenancy.routing`

Complete Reference — Architecture, Usage, Implementation & Migrations
plugins/Tenancy • v1.0.0 • Generated 2026-06-23

Table of Contents

1. 1. What the Tenancy Module Is
2. 2. The Two Planes: Control vs. Data
3. 3. Architecture & Request Lifecycle
4. 4. Component Inventory
5. 5. The Migrations Explained (one by one)
6. 6. Connection Routing: Registry, Resolver & Circuit Breaker
7. 7. The Core Flows (selection, invitations, refresh)
8. 8. How to Install & Wire It
9. 9. How to Use It (API + code)
10. 10. CLI: Provisioning & Fleet Migration
11. 11. How to Implement Against It (building tenant features)
12. 12. Configuration Reference
13. 13. Security Guarantees & Isolation
14. 14. Pitfalls & Rules

1. What the Tenancy Module Is

Tenancy is a first-party plugin (`plugins/Tenancy` , namespace `Plugins\Tenancy\`) that adds **database-per-tenant** multi-tenancy to the platform. It maps the authenticated `Identity.tenantId` to a fully *isolated* tenant database and rebinds the kernel's `DatabasePort` for that request — so every repository downstream transparently talks to the right tenant DB without knowing tenancy exists.

Property	Value
<code>solves</code>	<code>tenancy.routing</code>
<code>requires</code>	<code>database.management</code> , <code>auth.identity</code> , <code>user.management</code>
<code>exposes</code>	<code>TenantRegistryContract</code> , <code>TenantConnectionResolverContract</code> , <code>MembershipServiceContract</code> , <code>InvitationServiceContract</code> , <code>RefreshTokenServiceContract</code>
Activation	Essential module (runs on every request)
Built on	<code>plugins/Database</code> <code>ConnectionManager</code>

Key idea: the *database itself* is the tenant boundary. Tenant tables carry **no `tenant_id` column** and **no auth tables** — isolation is physical, not a `WHERE tenant_id = ?` filter. This eliminates the single largest class of multi-tenant bug: the forgotten tenant scope.

2. The Two Planes: Control vs. Data

Tenancy splits the world into two database planes:

Plane	Database	Tables	Connected
Control (central)	One, shared	<code>users</code> (from <code>plugins/User</code>), <code>tenants</code> , <code>user_tenants</code> , <code>tenant_invitations</code> , <code>refresh_tokens</code> , <code>audit_log</code>	Always
Data (per tenant)	One <i>per tenant</i>	Pure business domain — <code>projects</code> , <code>tasks</code> , ... (NO auth, NO <code>tenant_id</code> column)	On demand

The control plane is the *authority*: who exists, which tenants exist, who belongs to which tenant, and at what role. The data plane holds only the tenant's own business records. Foreign keys never cross databases — a user reference inside a tenant DB stores the central `users.user_id` ULID as an **opaque value** (no cross-DB FK).

3. Architecture & Request Lifecycle

Tenancy follows GDA's five-layer access rules. The runtime magic happens in a single HTTP pipeline stage, `TenantContextStage` , hooked at `after.load` with `priority: 5` (ahead of declarative route filters), so the request container exists and the `Identity` is already attached when it runs.

Request

- ▼ SecurityStage — Auth layer verifies JWT, sets Identity.tenantId from the signed "tnt" claim
- ▼ ResolveStage / LoadStage — builds the request ModuleContainer (central DatabasePort bound)
- ▼ after.load
 - | TenantContextStage (priority 5)
 - | tenantId = Identity.tenantId
 - | empty → no rebind (central stays bound)
 - | present → resolver.for(tenantId) → DatabasePort
 - | container.instance(DatabasePort::class, \$db)
- ▼ RouteFilterStage (auth, throttle, ...)
- ▼ ExecuteStage — controller → service → repository → DatabasePort (now the TENANT db)
- ▼ on success: resolver.recordSuccess(); on PDOException: resolver.recordFailure() (breaker)
- ▼ Response

Routing decision (fail-safe): a guest / unscoped request (empty `tenantId`) keeps the **central DatabasePort** — correct for login, the tenant picker, and public pages. Tenant-scoped controllers must require an `Identity` (the `auth` filter), so an unscoped request can never read tenant data.

4. Component Inventory

Layer	Class	Role
Provider	<code>Provider</code>	Wires all bindings; hooks <code>TenantContextStage</code> ; registers CLI commands
HTTP stage	<code>TenantContextStage</code>	Per-request tenant DB rebind + breaker feedback
Infrastructure	<code>TenantConnectionResolver</code>	<code>tenantId</code> → <code>DatabasePort</code> ; fail-closed; circuit breaker
Persistence	<code>TenantRegistry</code>	Reads central <code>tenants</code> table (cached, central connection only)
Persistence	<code>MembershipRepository</code>	Reads/writes <code>user_tenants</code>
Persistence	<code>InvitationRepository</code>	Reads/writes <code>tenant_invitations</code>
Persistence	<code>RefreshTokenRepository</code>	Reads/writes <code>refresh_tokens</code>
Audit	<code>AuditTrail</code>	Append-only writes to <code>audit_log</code>
Service	<code>MembershipService</code>	Tenant-selection flow (list + select + token mint)
Service	<code>InvitationService</code>	invite / accept / revoke
Service	<code>RefreshTokenService</code>	issue / rotate / revoke long-lived sessions
Controllers	<code>TenantController</code> , <code>InvitationController</code> , <code>AuthTokenController</code>	Thin HTTP boundaries
CLI	<code>CreateTenantCommand</code> , <code>MigrateTenantsCommand</code>	Provisioning & fleet migration
Domain	<code>Tenant</code> , <code>Membership</code> , <code>Invitation</code> , <code>RefreshTokenRecord</code> + status VOs	Entities & value objects

5. The Migrations Explained (one by one)

There are **two migration sets**. The **central migrations** (`database/migrations/`) run once against the control-plane DB via `hkm migrate:run`. The **tenant template** (`database/tenant-template/`) runs *inside every tenant database* via `hkm tenants:migrate`. All use LetMigrate's fluent `Blueprint` API.

5.1 Central — `create_tenants_table` (000001)

The **registry**: the source of truth mapping a `tenant_id` to the connection coordinates of its isolated database. Runs only on the central connection.

Column	Purpose
<code>tenant_id</code> char(31), unique	Public ULID/UUID identifier used everywhere (the <code>tnt</code> claim)
<code>name</code> , <code>slug</code> (unique)	Human label + DNS/db-safe slug (<code>^[a-z0-9-]+\$</code>)
<code>db_driver</code> , <code>db_host</code> , <code>db_port</code> , <code>db_name</code> , <code>db_username</code>	Connection coordinates — supports many-DBs-one-server <i>and</i> cross-server sharding
<code>db_password_enc</code> text	Connection password encrypted via EncryptionPort . Plaintext must never land here
<code>db_shard</code> nullable	Logical shard/cluster id for ops
<code>status</code> tinyint	1=active, 2=provisioning, 3=suspended, 4=deleted — drives fail-closed routing
<code>schema_version</code>	Last applied tenant migration batch — fleet drift visibility
<code>plan</code> , <code>trial_ends_at</code> , <code>settings</code> (json)	Optional subscription metadata
<code>version</code>	Optimistic-lock version
<code>created_at/updated_at</code> + <code>softDeletes()</code>	Timestamps; soft-delete so a deleted tenant is recoverable/auditable

Indexes: unique `tenant_id` & `slug` ; `status` ; (`db_shard` , `status`) . Engine InnoDB, utf8mb4, DYNAMIC row format.

5.2 Central — `create_user_tenants_table` (000002)

The **membership** M:N table: one central user can belong to many tenants, each with a tenant-scoped role and status. This is the *authority* the Auth layer consults at selection time and re-checks per request, so a revoked seat loses access before the JWT expires.

Column	Purpose
<code>user_id</code> , <code>tenant_id</code> char(31)	The pair; unique together (<code>uniq_user_tenant</code>)
<code>role</code>	<code>owner</code> <code>admin</code> <code>member</code> <code>viewer</code>
<code>status</code> tinyint	1=active, 2=invited, 3=suspended (only <i>active</i> is routable)
<code>joined_at</code>	When membership became active

Indexes: (`tenant_id` , `status`) "list members of a tenant"; (`user_id` , `status`) "list my tenants" (login hot path). FKs to central `users` and `tenants` — valid because both live in the same central DB (FKs never span databases).

5.3 Central — `create_tenant_invitations_table` (000003)

Email-based onboarding. It bridges "invited" and "has an account": membership needs a `user_id` , an invitation does not. Accepting converts the invite into a `user_tenants` row.

Column	Purpose
<code>invite_id</code> (unique)	Internal id of the invitation
<code>tenant_id</code> , <code>email</code> , <code>role</code>	Who is invited where, at what role
<code>token_hash</code> char(64), unique	SHA-256 of the token only — the raw token lives only in the emailed link
<code>invited_by</code>	Central <code>users.user_id</code> of the inviter
<code>status</code>	1=pending, 2=accepted, 3=revoked, 4=expired
<code>expires_at</code> , <code>accepted_at</code>	Lifecycle timestamps

Indexes support "one live invite per (tenant,email)" (enforced in the service) and email lookups. FK to `tenants` only — invitees may not yet exist in `users` .

5.4 Central — `create_refresh_tokens_table` (000004)

Stateless access JWTs are short (~15 min); this is their **revocable long-lived** counterpart. Only the SHA-256 of the token is persisted.

Column	Purpose
<code>token_id</code> (unique)	Internal id
<code>user_id</code>	Owner (FK to central <code>users</code>)
<code>token_hash</code> char(64), unique	SHA-256 of the refresh token — never raw
<code>tenant_id</code> nullable	<code>null</code> = not yet tenant-scoped (issued at login before selection)
<code>device</code> , <code>ip</code>	UA/device label and origin for session lists
<code>expires_at</code> , <code>revoked_at</code> , <code>last_used_at</code>	Lifecycle & rotation tracking

Index (`user_id` , `revoked_at`) = "list active sessions for a user" / revoke-all.

5.5 Central — `create_audit_log_table` (000005)

Append-only attributable trail for security/compliance: `login`, `tenant.switch`, `tenant.create`, `member.invite`, `member.join`, `auth.refresh`, etc. Lives central so cross-tenant admin actions are captured in one place. Stores identifiers + structured `meta` (json) only — **no passwords, tokens, or PII payloads**.

Indexes: `(tenant_id, occurred_at)`, `(user_id, occurred_at)`, `(action, occurred_at)` for time-ordered queries by tenant, user, or action.

5.6 Tenant template — `create_projects_table` (000001)

Applied inside every tenant database. Pure business domain: **no users table, no auth, no `tenant_id` column** — the database *is* the tenant boundary. `owner_user_id` stores the central ULID as an opaque value (no cross-DB FK).

5.7 Tenant template — `create_tasks_table` (000002)

Demonstrates that **tenant-local FKs are fine**: `tasks.project_id` → `projects.project_id` because both live in the same tenant DB. `assignee_user_id` is again an opaque central ULID.

Migration rules (LetMigrate, never Laravel/Doctrine): every migration has a matching `down()`; money uses `decimal`, never float; never mutate an applied migration — add a new one. `onUpdateCurrentTimestamp()` on PostgreSQL is auto-translated to a BEFORE UPDATE trigger by LetMigrate.

6. Connection Routing: Registry, Resolver & Circuit Breaker

6.1 TenantRegistry

Reads the central `tenants` table through `DatabasePort` only (it is a repository). The injected port is the **central** connection (`ConnectionManager::default()`) — it never resolves a tenant connection itself (that would be circular). Lookups are cached in `CachePort` with a short TTL; a miss (unknown tenant) is cached as a sentinel to absorb lookup storms for non-existent ids.

6.2 TenantConnectionResolver

The heart of routing: `for(string $tenantId): DatabasePort`.

```
public function for(string $tenantId): DatabasePort
{
    $name = 'tenant:' . $tenantId;

    if ($this->connections->has($name)) {           // already open this worker – reuse
        $this->guardBreaker($tenantId);
        return $this->connections->connection($name);
    }

    $tenant = $this->registry->find($tenantId)
        ?? throw UnknownTenantException::for($tenantId);    // FAIL CLOSED

    $this->guardStatus($tenant);    // suspended/deleted/provisioning → throw
    $this->guardBreaker($tenantId); // breaker open → throw

    $this->connections->register($name, $this->configFor($tenant)); // decrypts password
    return $this->connections->connection($name);
}
```

- **Fail closed:** unknown / suspended / deleted / unreachable → throw. Never falls back to another tenant or to central.
- **Lazy & reused:** the ConnectionManager builds the adapter lazily and caches it, so sockets open on first query and are reused within a worker.
- **Password decryption:** `configFor()` calls `EncryptionPort::decryptString()` to turn `db_password_enc` into a live config (MySQL or PostgreSQL).

6.3 Per-tenant circuit breaker

`TenantContextStage` reports query outcomes back to the resolver. After `TENANCY_BREAKER_THRESHOLD` consecutive failures the breaker *opens* for `TENANCY_BREAKER_COOLDOWN` seconds — one dead tenant DB fast-fails instead of hanging the worker pool, isolating it from the rest of the fleet.

```
// in TenantContextStage::handle()
try {
  $response = $next($request);
} catch (\PDOException $e) {
  $this->resolver->recordFailure($tenantId, $e); // increments counter; may trip breaker
  throw $e;
}
$this->resolver->recordSuccess($tenantId); // clears the counter on a healthy request
```

Failure responses from the stage map cleanly: **404** (unknown tenant), **403** (suspended), **410** (deleted), **503 + Retry-After: 30** (breaker open / provisioning).

7. The Core Flows

7.1 Tenant selection (MembershipService)

Turns an authenticated but *unscoped* user into a tenant-scoped session.

```
GET /api/me/tenants → active seats the user may switch into
POST /api/tenants/{tenantId}/select → re-mint a tenant-scoped token
```

`selectTenant()` **re-verifies** membership against central `user_tenants` (never trusts a client-supplied id), mints a JWT with the `tnt` claim via the Auth module, and audits `tenant.switch`. A revoked/suspended seat fails with `403` (audited `tenant.switch_denied`).

```
$selection = $memberships→selectTenant($identity→userId, $tenantId, $request→ip());
// → { token, tokenType: "Bearer", tenantId, role, expiresIn }
```

7.2 Invitations (InvitationService)

```
$res = $invitations→invite($tenantId, 'alice@example.com', 'member', $inviterUserId);
// → InvitationResult{ token, ... } — embed $res→token in the emailed link (shown ONCE)

$tenantId = $invitations→accept($rawToken, $identity→userId, $userVerifiedEmail, $ip);
$invitations→revoke($rawToken);
```

- Only the SHA-256 of the token is stored.
- `accept()` **requires** the authenticated user's verified email to match the invited address — an invite for `alice@` cannot be claimed by `bob@` (uses `hash_equals`).
- Idempotent on the seat (upsert) so a double-click can't fail; audits `member.join`.

Endpoint: `POST /api/invitations/accept { "token": "..." }` (behind `auth`). The email is read from the User identity store via `UserServiceContract` — never the request body. This is why Tenancy **requires:** `["user.management"]`.

7.3 Refresh tokens (RefreshTokenService)

```
$issued = $refreshTokens→issue($userId, $tenantId, $device, $ip); // raw token shown once
$rot     = $refreshTokens→rotate($rawToken, $ip);
// → RefreshRotation{ accessToken, expiresIn, refreshToken, refreshExpiresAt, tenantId, role
$refreshTokens→revoke($rawToken); // logout this session
$refreshTokens→revokeAllForUser($userId); // logout everywhere
```

- **One-time use:** `rotate()` revokes the presented token *before* issuing a new one — a captured/reused token then fails as already-revoked.
- For a tenant-scoped token, rotation **re-checks** `user_tenants` — a revoked seat can't refresh back in.
- Mints a fresh access JWT in the same call, so the client gets a ready-to-use credential.

Endpoints (UNAUTHENTICATED — the token IS the credential): `POST /api/auth/refresh` , `POST /api/auth/logout` .
There is no public "mint a refresh token" endpoint by design — `issue()` is called from your Auth flow at login / after select.

8. How to Install & Wire It

Step 1 — Run central migrations

```
hkm migrate:run    # creates tenants, user_tenants, tenant_invitations,
                  # refresh_tokens, audit_log on the CENTRAL connection
                  # (users lives in plugins/User)
```

Step 2 — Register as an ESSENTIAL module

In `projects/<name>/bootstrap/app.php` so every request is routed:

```
→withEssentialModules([
    \Plugins\Database\Provider::class,    // database.management (required dep)
    \Plugins\User\Provider::class,        // user.management (required dep)
    \Plugins\Auth\Provider::class,        // auth.identity (required dep)
    \Plugins\Tenancy\Provider::class,     // tenancy.routing
    // Crypto (EncryptionPort) and RedisCache (CachePort) must also be available
])
```

`EncryptionPort` (Crypto — decrypts tenant DB passwords) and `CachePort` (RedisCache — registry cache + breaker state) are hard dependencies of the registry/resolver. Wire both, or registration fails.

Step 3 — Mint a tenant-scoped Identity in your Auth layer

After the user selects a tenant, re-check `user_tenants` and put the tenant in the JWT `tnt` claim. The Auth security layer (`JwtAuthLayer`) reads it and sets `Identity.tenantId` ; `TenantContextStage` does the rest. **Re-check membership on every request** in the Auth layer so a revoked seat drops access before the JWT expires.

9. How to Use It (API + code)

Method & path	Auth	Body	Returns
GET /api/me/tenants	auth	—	{ data: [TenantSummary...] }
POST /api/tenants/{tenantId}/select	auth	—	{ token, tokenType, tenantId, role, expiresIn }
POST /api/invitations/accept	auth	{ token }	{ tenantId }
POST /api/auth/refresh	none	{ token }	{ accessToken, expiresIn, refreshToken, refreshExpiresAt, tenantId, role }
POST /api/auth/logout	none	{ token }	204

Consuming a contract from another plugin (inject the published interface, never internals):

```
// in your module.json
{ "requires": ["tenancy.routing"] }

// in a service/controller
public function __construct(
    private readonly MembershipServiceContract $memberships,
) {}

$tenants = $this->memberships->myTenants($identity->userId);
if ($this->memberships->isActiveMember($userId, $tenantId)) { /* ... */ }
```

10. CLI: Provisioning & Fleet Migration

tenants:create — provision a tenant

```
hkm tenants:create --name="Acme" --slug=acme \
  --db-name=tnt_acme --db-user=acme --db-password=secret \
  --db-host=127.0.0.1 --db-port=3306 [--driver=mysql|pgsql] [--template=PATH]
```

Four steps with compensating status (a half-provisioned tenant is detectable):

1. Insert a **tenants** row, **status=provisioning**, password **encrypted**.
2. **CREATE DATABASE IF NOT EXISTS** (idempotent; db-name validated `^[A-Za-z0-9_]+$`).
3. Run tenant template migrations against the new DB (via **MigrationServiceFactory**).
4. Flip **status=active** and stamp **schema_version**.

If step 2/3 fails the row stays `provisioning` (never `active`), so the resolver fast-fails routing to it and ops can retry. Slug validated `^[a-z0-9-]+$`.

`tenants:migrate` — fleet migration

```
hkm tenants:migrate          # apply template migrations to all ACTIVE tenants
hkm tenants:migrate --tenant=<id> # one tenant
hkm tenants:migrate --pretend    # print SQL, change nothing
```

- Each tenant DB keeps its own `let_migrations` table — a tenant mid-rollout is never corrupted by a global counter.
- After each success the central `tenants.schema_version` is stamped (drift visibility).
- **Failure isolation:** a failing tenant is logged and *skipped*; the run continues and is resumable. Exit code is FAILURE if any tenant failed.

11. How to Implement Against It (building tenant features)

The beauty of database-per-tenant: your tenant-domain code is **tenancy-agnostic**. You write an ordinary GDA module/plugin whose repositories use `DatabasePort` — by the time `ExecuteStage` runs, that port is already the tenant DB.

```
// A tenant-domain repository – NO tenant_id anywhere, NO awareness of tenancy
final class ProjectRepository
{
    public function __construct(private readonly DatabasePort $db) {}

    public function find(string $projectId): ?array
    {
        // $this->db is the TENANT database for this request – physical isolation
        return $this->db->queryOne(
            'SELECT * FROM projects WHERE project_id = :id AND deleted_at IS NULL',
            ['id' => $projectId],
        );
    }
}
```

Contrast with row-scoped multi-tenancy: there is **no** `WHERE tenant_id = ?` to forget. The forgotten-scope bug class is structurally impossible.

Checklist to add a new tenant table

1. Add a migration to `database/tenant-template/` (no `tenant_id`, no auth tables; user refs are opaque ULIDs).
2. Run `hkm tenants:migrate` to roll it out to the fleet.
3. Write repositories against `DatabasePort` as usual.
4. Tenant-local FKs are fine; cross-DB FKs are forbidden.

Reading the tenant in app code

```
$tenantId = $request->identity()?->tenantId ?? ''; // '' = unscoped (central)
// authorization still keys on (userId, tenantId, role/permission) – the tnt claim is a hint,
```


12. Configuration Reference (`module.json` / `env`)

Env	Default	Meaning
<code>TENANCY_MODE</code>	<code>tenant</code>	<code>legacy</code> <code>dual-write</code> <code>tenant</code> migration phases
<code>TENANCY_REGISTRY_TTL</code>	<code>60</code>	Registry cache TTL (seconds)
<code>TENANCY_BREAKER_THRESHOLD</code>	<code>5</code>	Failures before the breaker opens
<code>TENANCY_BREAKER_COOLDOWN</code>	<code>30</code>	Breaker open window (seconds)
<code>TENANCY_TEMPLATE_PATH</code>	<code>bundled</code>	Tenant template migrations path
<code>TENANCY_TOKEN_TTL</code>	<code>3600</code>	Scoped-token lifetime from <code>selectTenant</code> (s)
<code>TENANCY_REFRESH_TTL</code>	<code>2592000</code> (30d)	Refresh token lifetime (s)
<code>TENANCY_ACCESS_TTL</code>	<code>900</code> (15m)	Access JWT lifetime minted on rotation (s)

13. Security Guarantees & Isolation

- **Physical isolation:** one database per tenant; no shared rows; the DB is the boundary.
- **Fail closed:** unknown / suspended / deleted / unreachable → throw. Never falls back to another tenant or central.
- **Circuit breaker:** one dead tenant DB fast-fails after N failures; the fleet stays healthy.
- **Encrypted credentials:** tenant DB passwords stored encrypted via `EncryptionPort`; decrypted only in-memory at connect.
- **Hash-only secrets:** invitation & refresh tokens stored as SHA-256; raw values shown once; compared with `hash_equals`.
- **Re-verification:** membership is re-checked at select, at refresh-rotate, and (in the Auth layer) per request — a revoked seat loses access before its JWT expires.
- **Swoole-safe:** the resolved tenant `DatabasePort` is bound into the per-request `ModuleContainer` and discarded on `reset()`; the tenant id rides on the immutable `Request` / `Identity`, never a static or `CoreContainer`.
- **Audit trail:** append-only, identifiers + meta only, no secrets/PII.

14. Pitfalls & Rules

- Do **not** read `$_SERVER` /host in a module to find the tenant — use `$request->identity()->tenantId`.
- Do **not** add a `tenant_id` column to tenant-template tables — the DB is the boundary.
- Do **not** create cross-database foreign keys — store the central ULID as an opaque value.
- Do **not** store plaintext tenant DB passwords — always encrypt via `EncryptionPort`.
- Do **not** store raw invitation/refresh tokens — hash only.
- Do **not** trust a client-supplied tenant id — `selectTenant` re-verifies against `user_tenants`.

- Do **not** bind `DomainContext` or the tenant into `CoreContainer` — Swoole leak.
- Do **not** use Laravel/Doctrine migrations — only LetMigrate, with a `down()` .

Swoole connection pooling (optional): by default `ConnectionManager` is request-scoped. For long-lived workers, bind the manager + resolver into the **CoreContainer** so tenant adapters persist per worker; cap with LRU eviction of idle tenant connections, and front the DB tier with ProxySQL/PgBouncer under PHP-FPM.